

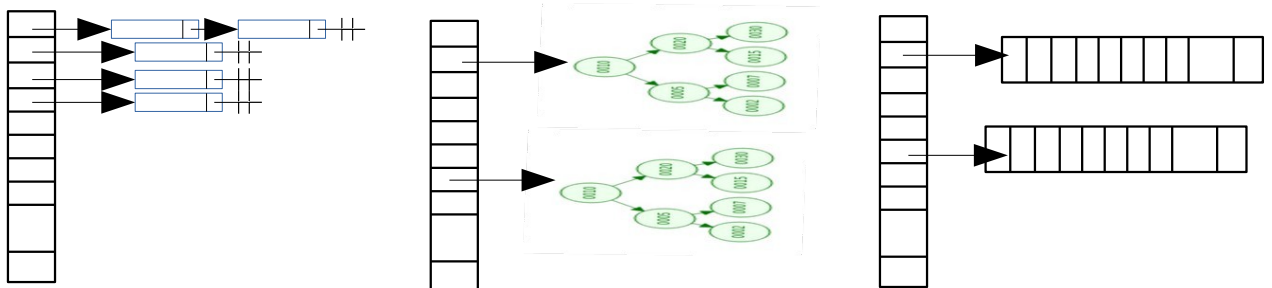
1) Sobre árvores binárias (0,5 pt cada)

- Retirada de um elemento de uma árvore **binária de busca**: para a retirada de um nó que tenha 2 filhos, qual é ou quais são os procedimentos básicos para realizar essa tarefa? Por quê?
- A busca em uma árvore **binária de busca** é geralmente melhor que em uma lista ordenada? Por quê?
- Quais são e para quê servem os algoritmos de caminhadas em árvores **binárias**?

2) Sobre LISTAS, PILHAS e FILAS (não confundir):

- Por que é recomendável usar um esquema de FILA Circular ao implementar uma fila FIFO? Também serve quando implementado com alocação dinâmica? (0,5 pt)
- Buscas em LISTAS não são muito eficientes (porque são buscas sequenciais, certo? - nada de busca binária aqui), então se formos obrigados a usar uma lista, como podemos oferecer a melhor busca possível? (0,5 pt)
- Uma PILHA é uma estrutura que pode ser usada para ‘inverter’ a ordem dos dados (0,5 pt)

3) Um hash tradicional é composto por uma tabela de espalhamento em um array de listas (ou seja, o chamado de Encadeamento direto, figura da esquerda). Imagine que, em vez de usar um array de listas, alguém resolve implementar como um array de árvores binárias de busca (figura do meio) e outro pensa em fazer um array de arrays (figura da direita). Faz sentido? Analise os aspectos que julgar importante. (3,5 pts.)



4) Considere que não sabemos quantos elementos há em uma lista **duplamente** encadeada (como da figura 1, mas queremos descobrir e acessar o elemento central dessa lista (que tem, por exemplo, 1001 elementos). Podemos usar a seguinte ideia: ter 2 iteradores, um que começa no início e avança em direção ao final e outro que vai do final em direção ao início e vamos avançando cada um dos iteradores – um de cada vez. Quando ambos estiverem no mesmo elemento, este será o elemento central – concorda? Implemente o **código deste procedimento** (use os nomes das classes, funções e variáveis, conforme exemplo abaixo) que, ao final, informa qual é o elemento central (3,5 pt)

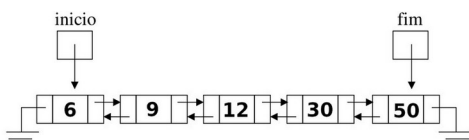


Figura 1: Exemplo de lista duplamente encadeada

```
class No:
    def __init__(self, d):
        self.dado = d
        self.prox = None
        self.ant = None
```

```
class lista_2mte:
    def __init__(self):
        self.inicio = None
        self.fim = None

    # exemplo de função de
    # inserção na frente
    def insereNaFrente(self, novoDd):
        novoNo = No(novoDd)
        novoNo.prox = self.inicio
        if self.inicio is not None:
            self.inicio.ant = novoNo
        self.inicio = novoNo
    .....
```